E4 Educator v2.0

T03

Buttons and debouncing

Tier 1 · Tutorial 3 of 7

Walark Electronics · Fundrum Engineering · May 2026

In this tutorial

You will learn:

- Why mechanical buttons bounce and what it looks like in firmware
- Why delay() debounce is not acceptable in real projects
- How to implement millis()-based debounce properly
- How to detect a clean single press event — not just a held state
- How to detect short press versus long press on the same button
- How to build a reusable Button class you can drop into any project
- How to implement a simple state machine driven by button events

You will need:

- E4 Educator v2.0 board with ESP32 fitted
- T01 and T02 completed
- USB-C cable (data-capable)

1 Why buttons bounce

A mechanical button is two metal contacts that physically touch when you press it. When they meet, they don't make clean contact immediately — they bounce against each other several times in the space of a few milliseconds before settling into a stable connection. This is called contact bounce, or simply bounce.

To a human, a button press feels instantaneous. But to a microcontroller running at 240MHz and checking the pin thousands of times per second, those few milliseconds of bouncing look like a rapid series of presses and releases — sometimes ten or more in a single physical press.

1.1 What bounce looks like in firmware

If you run this code and watch the Serial monitor, you'll see the problem clearly:

```
#include <Arduino.h>

int pressCount = 0;

void setup() {
  Serial.begin(115200);
  pinMode(BTN_NEXT, INPUT_PULLUP);
  Serial.println("Press NEXT button once and watch...");
}

void loop() {
  if (digitalRead(BTN_NEXT) == LOW) {
    pressCount++;
    Serial.printf("LOW detected! Count: %d\n", pressCount);
  }
}
```

Upload this, open the Serial monitor, and press NEXT once. You'll likely see something like:

```
LOW detected! Count: 1
LOW detected! Count: 2
LOW detected! Count: 3
LOW detected! Count: 4
LOW detected! Count: 5
...
```

One physical press, multiple detections. This is bounce — and it makes button-driven firmware unreliable without a debounce strategy.

2 The delay() bodge — and why it fails

The most common beginner fix is to add delay(200) after detecting a press. This works for simple cases but fails as soon as your firmware needs to do anything else simultaneously.

```
// The delay() bodge — do not use in real projects
void loop() {
```

```
  if (digitalRead(BTN_NEXT) == LOW) {
    pressCount++;
    Serial.printf("Pressed! Count: %d\n", pressCount);
    delay(200);   // Blocks everything for 200ms
  }
}
```

Key point

delay(200) inside a button check blocks ALL other firmware for 200 milliseconds on every single press. If you're also blinking LEDs, updating a display, or reading sensors — all of that stops dead while you wait. The millis() approach eliminates this completely.

3 millis() debounce — the right approach

The correct approach is to track when the pin state last changed and only act on it once it has been stable for a defined period — the debounce interval. This is done with millis() so nothing is blocked.

3.1 The debounce logic

The pattern has four components:

- rawState — what digitalRead() returns right now
- stableState — the last confirmed stable state after debounce
- lastChangeTime — when the pin state last changed
- DEBOUNCE_MS — how long the pin must be stable before we accept the change

```
#include <Arduino.h>

const unsigned long DEBOUNCE_MS = 50;  // 50ms is reliable for most buttons

bool rawState      = HIGH;  // What we read right now
bool stableState   = HIGH;  // Last confirmed stable state
bool lastRawState  = HIGH;  // Previous raw state for change detection
unsigned long lastChangeTime = 0;

int pressCount = 0;

void setup() {
  Serial.begin(115200);
  pinMode(BTN_NEXT, INPUT_PULLUP);
  Serial.println("T03 — millis() debounce demo");
}

void loop() {
  unsigned long now = millis();
  rawState = digitalRead(BTN_NEXT);

  // Has the raw state changed since last loop?
  if (rawState != lastRawState) {
    lastChangeTime = now;     // Reset the stability timer
    lastRawState = rawState;
  }

  // Has the pin been stable for long enough?
  if ((now - lastChangeTime) >= DEBOUNCE_MS) {
    if (rawState != stableState) {
      stableState = rawState;

      // Act on the newly stable state
      if (stableState == LOW) {
        pressCount++;
        Serial.printf("Clean press! Count: %d\n", pressCount);
      }
    }
  }
```

```
}
```

Upload this and press NEXT repeatedly. Each physical press now registers exactly once, no matter how bouncy the contact. And because there is no delay(), everything else in loop() continues to run freely during the debounce window.

3.2 Choosing the debounce interval

Interval	When to use it
`20ms`	High quality tactile switches with low bounce. Responsive feel.
`50ms`	Standard for most through-hole and SMD tactile buttons. Good balance of responsiveness and reliability. Use this as your default.
`100ms`	Older or cheaper switches, toggle switches, reed relays. Adds noticeable lag but eliminates all bounce.
`200ms+`	Avoid — this is the delay() territory. The response feels sluggish and this is the equivalent of the bodge.

4 Detecting a clean press event

The debounce code above tells you the stable state of the button. But in most real firmware, you don't want to know "is this button currently held down?" — you want to know "did the user just press this button?" — a single event at the moment of press.

This is the difference between a state and an event. The transition from HIGH to LOW (released to pressed) is the event you want to detect.

4.1 Edge detection

A falling edge is the transition from HIGH to LOW — the moment the button is pressed. A rising edge is LOW to HIGH — the moment it is released. For most UI interactions, you want the falling edge.

```
#include <Arduino.h>

const unsigned long DEBOUNCE_MS = 50;

bool rawState     = HIGH;
bool stableState  = HIGH;
bool lastRawState = HIGH;
bool prevStable   = HIGH;   // Previous stable state for edge detection
unsigned long lastChangeTime = 0;

int pressCount = 0;

// Returns true ONCE on the falling edge (press)
bool buttonPressed() {
  unsigned long now = millis();
  rawState = digitalRead(BTN_NEXT);

  if (rawState != lastRawState) {
    lastChangeTime = now;
    lastRawState = rawState;
  }

  if ((now - lastChangeTime) >= DEBOUNCE_MS) {
    if (rawState != stableState) {
      prevStable   = stableState;
      stableState  = rawState;

      // Falling edge: HIGH -> LOW = press event
      if (prevStable == HIGH && stableState == LOW) {
        return true;
      }
    }
  }
  return false;
}

void setup() {
  Serial.begin(115200);
  pinMode(BTN_NEXT, INPUT_PULLUP);
```

```
  Serial.println("T03 — edge detection demo");
}

void loop() {
  if (buttonPressed()) {
    pressCount++;
    Serial.printf("Press event! Count: %d\n", pressCount);
  }
  // Everything else in loop() runs freely
}
```

buttonPressed() now returns true for exactly one loop() pass at the moment of press, then false until the next press. This is the clean event-driven pattern used in every Fundrum project.

5 Short press and long press

With only two buttons on the E4 (NEXT and ENT), being able to distinguish a short press from a long press effectively doubles your available user inputs. This is a very common pattern in embedded UI design.

The approach: record the time when the button is pressed. When it is released, measure how long it was held. Under a threshold = short press. Over the threshold = long press.

```
#include <Arduino.h>

const unsigned long DEBOUNCE_MS  = 50;
const unsigned long LONG_PRESS_MS = 800;  // Hold for 800ms = long press

// Per-button state
struct Button {
  int pin;
  bool raw, stable, lastRaw, prevStable;
  unsigned long lastChange, pressedAt;
  bool isHeld;
};

Button btnNext = { BTN_NEXT, HIGH, HIGH, HIGH, HIGH, 0, 0, false };
Button btnEnt  = { BTN_ENT,  HIGH, HIGH, HIGH, HIGH, 0, 0, false };

// Returns 0=none, 1=short press, 2=long press
int readButton(Button &btn) {
  unsigned long now = millis();
  btn.raw = digitalRead(btn.pin);

  if (btn.raw != btn.lastRaw) {
    btn.lastChange = now;
    btn.lastRaw = btn.raw;
  }

  if ((now - btn.lastChange) >= DEBOUNCE_MS) {
    if (btn.raw != btn.stable) {
      btn.prevStable = btn.stable;
      btn.stable = btn.raw;

      if (btn.prevStable == HIGH && btn.stable == LOW) {
        // Falling edge — button just pressed
        btn.pressedAt = now;
        btn.isHeld = true;
      }

      if (btn.prevStable == LOW && btn.stable == HIGH) {
        // Rising edge — button just released
        btn.isHeld = false;
        unsigned long heldFor = now - btn.pressedAt;
        if (heldFor >= LONG_PRESS_MS) {
          return 2;  // Long press
        } else {
          return 1;  // Short press
```

```
        }
      }
    }
  }
  return 0;  // No event
}

void setup() {
  Serial.begin(115200);
  pinMode(BTN_NEXT, INPUT_PULLUP);
  pinMode(BTN_ENT,  INPUT_PULLUP);
  Serial.println("T03 — short/long press demo");
  Serial.println("Short press NEXT, long press NEXT, then try ENT");
}

void loop() {
  int nextResult = readButton(btnNext);
  int entResult  = readButton(btnEnt);

  if (nextResult == 1) Serial.println("NEXT: short press");
  if (nextResult == 2) Serial.println("NEXT: LONG press");
  if (entResult  == 1) Serial.println("ENT:  short press");
  if (entResult  == 2) Serial.println("ENT:  LONG press");
}
```

Key point

With short and long press on both NEXT and ENT, you have four distinct input events from two physical buttons. This is enough to build a complete multi-screen menu system — which is exactly what you'll do in Tier 2.

6 A reusable Button class

The struct-based approach works well. Let's go one step further and wrap it in a clean C++ class that can be dropped into any project with no modification. This is the Button implementation used in all E4 Educator projects from T03 onwards.

Create a new file in your project's src/ folder called Button.h:

```
// Button.h — Fundrum E4 Educator reusable debounce class
// Drop this into src/ and #include "Button.h"
#pragma once
#include <Arduino.h>

class Button {
public:
  enum Event { NONE, SHORT_PRESS, LONG_PRESS };

  Button(int pin,
         unsigned long debounceMs  = 50,
         unsigned long longPressMs = 800)
    : _pin(pin), _debounceMs(debounceMs), _longPressMs(longPressMs),
      _raw(HIGH), _stable(HIGH), _lastRaw(HIGH), _prevStable(HIGH),
      _lastChange(0), _pressedAt(0) {}

  void begin() {
    pinMode(_pin, INPUT_PULLUP);
  }

  Event read() {
    unsigned long now = millis();
    _raw = digitalRead(_pin);

    if (_raw != _lastRaw) {
      _lastChange = now;
      _lastRaw    = _raw;
    }

    if ((now - _lastChange) >= _debounceMs) {
      if (_raw != _stable) {
        _prevStable = _stable;
        _stable     = _raw;

        if (_prevStable == HIGH && _stable == LOW) {
          _pressedAt = now;
        }

        if (_prevStable == LOW && _stable == HIGH) {
          unsigned long held = now - _pressedAt;
          return (held >= _longPressMs) ? LONG_PRESS : SHORT_PRESS;
        }
      }
    }
    return NONE;
```

```
  }

  bool isHeld() {
    return (_stable == LOW);
  }

private:
  int _pin;
  unsigned long _debounceMs, _longPressMs;
  bool _raw, _stable, _lastRaw, _prevStable;
  unsigned long _lastChange, _pressedAt;
};
```

Now main.cpp becomes very clean:

```
#include <Arduino.h>
#include "Button.h"

Button nextBtn(BTN_NEXT);
Button entBtn(BTN_ENT);

void setup() {
  Serial.begin(115200);
  nextBtn.begin();
  entBtn.begin();
  Serial.printf("T03 — Button class demo  FW: %s\n", FW_VERSION);
}

void loop() {
  Button::Event nextEvent = nextBtn.read();
  Button::Event entEvent  = entBtn.read();

  if (nextEvent == Button::SHORT_PRESS) Serial.println("NEXT: short press");
  if (nextEvent == Button::LONG_PRESS)  Serial.println("NEXT: LONG press");
  if (entEvent  == Button::SHORT_PRESS) Serial.println("ENT:  short press");
  if (entEvent  == Button::LONG_PRESS)  Serial.println("ENT:  LONG press");
}
```

Keep Button.h

Save this file — you will use it in every tutorial from here on. From T04 onwards, Button.h is part of the standard E4 project template. Copy it into the src/ folder of each new project.

7 Practical exercise — a button-driven state machine

This exercise puts everything together. The three LEDs cycle through states driven entirely by clean, debounced button events — no delay() anywhere.

NEXT short press advances the active LED. ENT short press toggles the active LED on/off. ENT long press turns all LEDs off and returns to state 0.

```
#include <Arduino.h>
#include "Button.h"

Button nextBtn(BTN_NEXT);
Button entBtn(BTN_ENT);

// State machine
enum AppState { STATE_RED, STATE_GREEN, STATE_BLUE };
AppState currentState = STATE_RED;

bool ledRed   = false;
bool ledGreen = false;
bool ledBlue  = false;

void applyLeds() {
  digitalWrite(LED_RED,   ledRed   ? HIGH : LOW);
  digitalWrite(LED_GREEN, ledGreen ? HIGH : LOW);
  digitalWrite(LED_BLUE,  ledBlue  ? HIGH : LOW);
}

void printState() {
  const char* names[] = { "RED", "GREEN", "BLUE" };
  Serial.printf("State: %s  R:%d G:%d B:%d\n",
    names[currentState], ledRed, ledGreen, ledBlue);
}

void setup() {
  Serial.begin(115200);
  Serial.printf("T03 Exercise — State Machine  FW: %s\n", FW_VERSION);

  pinMode(LED_RED,   OUTPUT);
  pinMode(LED_GREEN, OUTPUT);
  pinMode(LED_BLUE,  OUTPUT);
  applyLeds();

  nextBtn.begin();
  entBtn.begin();

  Serial.println("NEXT: cycle LED  ENT short: toggle  ENT long: reset");
  printState();
}

void loop() {
  Button::Event nextEv = nextBtn.read();
  Button::Event entEv  = entBtn.read();
```

```
  // NEXT short press — advance to next LED
  if (nextEv == Button::SHORT_PRESS) {
    currentState = (AppState)((currentState + 1) % 3);
    printState();
  }

  // ENT short press — toggle active LED
  if (entEv == Button::SHORT_PRESS) {
    switch (currentState) {
      case STATE_RED:   ledRed   = !ledRed;   break;
      case STATE_GREEN: ledGreen = !ledGreen; break;
      case STATE_BLUE:  ledBlue  = !ledBlue;  break;
    }
    applyLeds();
    printState();
  }

  // ENT long press — reset all
  if (entEv == Button::LONG_PRESS) {
    currentState = STATE_RED;
    ledRed = ledGreen = ledBlue = false;
    applyLeds();
    Serial.println("Reset!");
    printState();
  }
}
```

7.1 What to observe

- Each NEXT short press moves to the next LED with no stutter or double-fire — clean debounce at work
- ENT short press reliably toggles the active LED, even pressed quickly after NEXT
- ENT long press resets everything — hold for just under 800ms and note it does NOT trigger long press
- The Serial monitor shows every state transition with LED states
- No delay() anywhere — the state machine is fully non-blocking

Try this

Change LONG_PRESS_MS to 1500 in Button.h. Notice how it now requires a longer hold before triggering. Then change it back to 800. This single constant controls the feel of every long press in every project that uses Button.h — the value lives in one place.

8 T03 summary

Concept	Key takeaway
Contact bounce	Mechanical buttons bounce for a few milliseconds. At 240MHz the ESP32 sees this as dozens of presses. Always debounce.
delay() debounce	Blocks all firmware for every press. Never acceptable in real projects. Replace with millis().

millis() debounce	Track when the pin last changed. Accept the new state only after it has been stable for DEBOUNCE_MS. Non-blocking.
Edge detection	Detect the transition HIGH→LOW (press) not just the LOW state. Returns true once per press, not continuously.
Short / long press	Record pressedAt on falling edge. On rising edge, measure held duration. Under threshold = short, over = long.
Button class	Button.h encapsulates all debounce logic. Drop it into any project. Call begin() in setup(), read() in loop().
State machines	Button events drive state transitions cleanly. enum + switch is the natural C++ pattern for firmware state machines.

What's next

- T04 — I2C fundamentals: with reliable button handling in place, it's time to read your first sensor. The AHT20 temperature and humidity module is already on the E4 board waiting to be used.
- T05 — OLED display: once you can read the AHT20, displaying its data on the OLED is the natural next step — your first complete sensor-to-display project.